First order logic or Predicate logic

- Predicate logic is a more powerful language for expressing a KB (variables and quantifiers)
- We will look at proof methods based on
  - Resolution (sound and complete for general clauses)
  - Forward chaining (sound and complete for Horn clauses) Used in deductive databases (DATALOG)
  - Backward chaining (sound and complete for Horn clauses) Used in Logic Programming (PROLOG).

TRANSFORMATION INTO CLAUSES (1)

- Any well formed formula (wff of predicate logic can be transformed into a set of general clauses.
- Steps
- 1) **Transformation into a closed wff**

Example the formula:

– $\forall X\ (p(Y) \rightarrow \sim(\forall Y\ (q(X,Y) \rightarrow p(Y))))$ (1)

is transformed into:

– $\forall X\ \forall Y\ (p(Y) \rightarrow \sim(\forall Y\ (q(X,Y) \rightarrow p(Y))))$ (2)

- 2) **Application of equivalences for logical connectives** (for example $A \rightarrow B$ is replaced by $\sim A \vee B$) and it is reduced to and-or form.

Formula (2) becomes:

– $\forall X\ \forall Y\ (\sim p(Y) \vee \sim(\forall Y\ (\sim q(X,Y) \vee p(Y))))$ (3)

TRANSFORMATION INTO CLAUSES (2)

- 3) **Applying negation to atoms rather than to compound formulas**, keeping in mind that:

  $\forall X \sim A$ is equivalent to $\sim \exists X\, A$

  $\exists X \sim A$ is equivalent to $\sim \forall X\, A$

  $\sim(A_1 \vee A_2 \vee \ldots \vee A_n)$ is equivalent to $\sim A_1 \wedge \sim A_2 \wedge \ldots \wedge \sim A_n$

  $\sim(A_1 \wedge A_2 \wedge \ldots \wedge A_n)$ is equivalent to $\sim A_1 \vee \sim A_2 \vee \ldots \vee \sim A_n$

  (De Morgan's laws).

  (3) $\forall X\, \forall Y\, (\sim p(Y) \vee \sim(\forall Y\, (\sim q(X,Y) \vee p(Y))))$ becomes:

  $\forall X\, \forall Y\, (\sim p(Y) \vee (\exists Y\, (q(X,Y) \wedge \sim p(Y))))$ (4)

- 4) **Renaming of variables**, in case of conflicts.

  in (4) the second variable Y is renamed Z:

  $\forall X\, \forall Y\, (\sim p(Y) \vee (\exists Z\, (q(X,Z) \wedge \sim p(Z))))$ (5)

TRANSFORMATION INTO CLAUSES (3)

- 5) **Moving the quantifiers** to the front of the formula (prenex form).

  $\forall X\ \forall Y\ \exists Z\ (\sim p(Y) \vee (q(X,Z) \wedge \sim p(Z)))$ (6)

- 6) **Conjunctive normal form** i.e., as a conjunction of disjunctions, with the quantifiers up front.

  $\forall X\ \forall Y\ \exists Z\ ((\sim p(Y) \vee q(X,Z)) \wedge (\sim p(Y) \vee \sim p(Z)))$ (7)

- 7) **Skolemization:** every existentially quantified variable is replaced by a function of the universally quantified variables that precede it. This function is called a Skolem function.

  For example, a formula of the form: $\forall X\ \exists Y\ p(X,Y)$ can equivalently be expressed as: $\forall X\ p(X,g(X))$

  In (7) Z is replaced by f(X,Y), because Z lies within the scope of the quantifiers $\forall X$ and $\forall Y$:

  $\forall X \forall Y\ ((\sim p(Y) \vee q(X,f(X,Y))) \wedge (\sim p(Y) \vee \sim p(f(X,Y))))$ (8)

TRANSFORMATION INTO CLAUSES (4)

- **Loss of expressiveness.** It is not the same to assert: F: $\exists$X p(X) or F′: p(f).
- However, the property still holds that F is inconsistent if and only if F′ is inconsistent.
- 8) **Elimination of universal quantifiers**: the result is a formula called universal (all its variables are universally quantified) in conjunctive normal form.
- $((\sim p(Y) \vee q(X,f(X,Y))) \wedge (\sim p(Y) \vee \sim p(f(X,Y))))$ (9)
- A formula of this kind represents **a set of clauses** (each given by a conjunct in the formula). The resulting clause normal form is:
  $\{\sim p(Y) \vee q(X,f(X,Y)), \sim p(Y) \vee \sim p(f(X,Y))\}$ (10)
- The second clause can be rewritten by renaming its variables (i.e., replacing the formula with a variant of it).
- $\{\sim p(Y) \vee q(X,f(X,Y)), \sim p(Z) \vee \sim p(f(W,Z))\}$ (11)

TRANSFORMATION INTO CLAUSES (5)

- Any first-order theory T can be transformed into a theory T′ in clause form.
- Even though T is not logically equivalent to T′ (because of the introduction of Skolem functions), the following property still holds:

Property

- Let T be a first-order theory and T′ a transformation of it into clauses. Then T is unsatisfiable if and only if T′ is unsatisfiable.
- The resolution principle is a proof procedure that works by contradiction and is based on the concept of unsatisfiability.

UNIFICATION

- To apply the resolution principle to non-"ground" clauses, it is necessary to introduce the concept of unification [Robinson 65].
- **Unification**: a formal manipulation procedure used to establish when two expressions can be made to coincide by applying suitable substitutions.
- **Substitution**: σ a set of bindings of terms T_i to variable symbols X_i (i=1,..,n).

  $\sigma = \{X_1/T_1, X_2/T_2,...,X_n/T_n\}$

  where $X_1, X_2,..., X_n$ are distinct.
- The substitution corresponding to the empty set is called the **identity substitution** (ε).

SUBSTITUTIONS AND RENAMING

- **Applying the substitution σ to an expression E**, $[E]\sigma$, produces a new expression obtained by simultaneously substituting each variable X_i of the expression with the corresponding term T_i. $[E]\sigma$ is called an **instance** of E.

- **Renaming**: substitutions that simply change the name of some of the variables of E. , $[E]\sigma$ is a **variant** of E.

SUBSTITUTIONS EXAMPLE

- Applying the substitution σ = {X/c, Y/a, Z/W} to the expression p(X,f(Y),b,Z) produces the instance p(c,f(a),b,W).

Similarly:

[c(Y,Z)]{Y/T, Z/black} =
c(T,black)

[c(T,black)]{Y/T, Z/black} = c(T,black)

[c(Y,Z)]{Y/white,T/white,Z/black}=

c(white,black)

[c(T,black)]{Y/white,T/white,Z/black}=

c(white,black)

[c(Y,Z)]{Y/T, Z/white} = c(T,white)

[c(T,black)]{Y/T, Z/white} = c(T,black)

[t(I(a),t(I(b),I(H)))]{H/I(a), Y/I(b),
Z/I(I(a))}=

t(I(a),t(I(b),I(I(a))))

[t(H,t(Y,Z))]{H/T,Y/X,Z/W}=t(T,t(X,W))

[c(T,black)]ε = c(T,black)

[p(X,Y)]{X/a,Y/X} =
p(a,X)

COMBINING SUBSTITUTIONS

- **Combinations of substitutions**:

  $\sigma_1=\{X_1/T_1,X_2/T_2,...,X_n/T_n\}\sigma_2=\{Y_1/Q_1,Y_2/Q_2,...,Y_m/Q_m\}$

- **composition** $\sigma_1\sigma_2$ of σ_1 and σ_2 is the substitution

  $\{X_1/[T_1]\sigma_2,...,X_n/[T_n]\sigma_2,Y_1/Q_1,Y_2/Q_2,..,Y_m/Q_m\}$

  deleting the pairs $X_i/[T_i]\sigma_2$ for which $X_i=[T_i]\sigma_2$ and the pairs Y_j/Q_j for which Y_j belongs to the set $\{X_1, X_2,...,X_n\}$.

Example: σ_1 = {X/f(Z), W/R, S/c} σ_2 = {Y/X, R/W,Z/b}

produces: $\sigma_3 = \sigma_1\sigma_2$ = {X/f(b), S/c, Y/X, R/W, Z/b}.

MORE GENERAL SUBSTITUTIONS

- **More general substitutions:** A substitution θ is more general than a substitution σ if there exists a substitution λ such that $\sigma=\theta\lambda$.

Example: The substitution $\theta = \{Y/T, Z/black\}$ is more general than the substitution $\sigma = \{Y/white, T/white, Z/black\}$ since σ can be obtained through the composition $\{Y/T, Z/black\}\{T/white\}$ ($\sigma=\theta\lambda$, and $\lambda = \{T/white\}$).

UNIFYING SUBSTITUTION

- **unification** makes two or more atoms (or terms) identical (or more precisely, their **instances**) through a suitable substitution.
- If only two atoms (or terms) are considered, one of which has no variables at all, this is a special case of unification, called **pattern-matching**.
- A set of atoms (or terms) $A_1, A_2, ..., A_n$ is **unifiable** if there exists a substitution σ such that:

  $[A_1]\sigma = [A_2]\sigma = ... = [A_n]\sigma.$

- The substitution σ is called a **unifying substitution** (or unifier)

EXAMPLE OF UNIFYING SUBSTITUTIONS (1)

Example Consider the atoms:

A1=c(Y,Z) A2=c(T,black)

- possible unifying substitutions are:

  θ = {Y/T, Z/black}

  σ = {Y/white, T/white, Z/black}

- applying them produces the same instance:

  [c(Y,Z)]θ = [c(T,black)]θ = c(T,black)

  [c(Y,Z)]σ = [c(T,black)]σ = c(white,black)

- The substitution: □

  λ={Y/T, Z/white} is not a unifier for A1 and A2 because it produces different instances.

EXAMPLE OF UNIFYING SUBSTITUTIONS (2)

- **For the atoms:**

  A3: p(X,X,f(a,Z)) A4: p(Y,W,f(Y,J))

- possible unifying substitutions are:

  μ = {X/a, Y/a, W/a, J/Z}

  φ = {X/a,Y/a,W/a, J/c, Z/c}

- applying them to A3 and A4 produces the same instance:

  p(a,a,f(a,Z)) in the case of substitution μ

  p(a,a,f(a,c)) in the case of substitution φ.

- Several unifying substitutions may exist; we want to find the most general one (mgu, **most general unifier**).

  μ mgu: φ is obtained from μ by composing it with: α = {Z/c}.

MGU EXAMPLES

θ denotes the most general unifying substitution:

1) p(X,f(a)) p(b, Y)

θ = {X/b, Y/f(a)}

2) t(l(a),t(l(b),l(c)))t(l(X),Z)

θ = {X/a, Z/t(l(b),l(c))}

The terms t(l(a),t(l(b),l(c))) and t(l(X),Z) represent binary trees.

3) t(l(a),t(l(b),l(H))) t(H,t(Y,Z))

θ = {H/l(a), Y/l(b), Z/l(l(a))}

4) f(X,g(Y),T) f(c(a,b),g(g(a,c)),Z).

θ = {X/c(a,b), Y/g(a,c), T/Z}

5) .(a,.(b,.(c,.(d,[])))) .(X,Y)

θ = {X/a, Y/.(b,.(c,.(d,[])))}

represent lists

6) .(a,.(b,.(c,.(d,[])))) .(X,.(Y,Z))

θ = {X/a, Y/b, Z/.(c,.(d,[]))}

7) .(a,.(b,.(c,.(d,[])))) .(c,Y)

They are not unifiable.

8) t(t(t(X,Y),l(c)),Y)
t(t(t(Y,Y),l(c)),l(X))

They are not unifiable

UNIFICATION ALGORITHM

- The Unify function takes as input parameters two atoms or terms to unify, A and B, and the substitution already applied so far.
- The function always terminates and can return either the most general substitution that unifies A and B, or a failure (FALSE).
- A compound term (i.e., other than a constant or a variable) is represented by an operator OP (the term's functional symbol) and, as its other elements, the term's arguments ARGS (a list).
- E.g.: f(a,g(b,c),X) represented as: [f,a,[g,b,c],X].
- The function FIRST applied to a list L returns the first element of L, while the function REST returns the rest of list L.

The unification algorithm

function UNIFY(x, y, θ) **returns** a substitution to make x and y identical
  inputs: x, a variable, constant, list, or compound
    y, a variable, constant, list, or compound
    θ, the substitution built up so far

  if θ = failure **then return** failure
  else if $x = y$ **then return** θ
  else if VARIABLE?(x) **then return** UNIFY-VAR(x, y, θ)
  else if VARIABLE?(y) **then return** UNIFY-VAR(y, x, θ)
  else if COMPOUND?(x) **and** COMPOUND?(y) **then**
    return UNIFY(ARGS[x], ARGS[y], UNIFY(OP[x], OP[y], θ))
  else if LIST?(x) **and** LIST?(y) **then**
    return UNIFY(REST[x], REST[y], UNIFY(FIRST[x], FIRST[y], θ))
  else return failure

From: S. Russell & P. Norvig: "Artificial Intelligence: A Modern Approach", Pearson ed.

The unification algorithm

function UNIFY-VAR(var, x, θ) **returns** a substitution

inputs: var, a variable

x, any expression

θ, the substitution built up so far

if $\{var/val\} \in \theta$ **then return** UNIFY(val, x, θ)
else if $\{x/val\} \in \theta$ **then return** UNIFY(var, val, θ)
else if OCCUR-CHECK?(var, x) **then return** failure
else return add $\{var/x\}$ to θ

From: S. Russell & P. Norvig: "Artificial Intelligence: A Modern Approach", Pearson ed.

OCCUR CHECK

- This is the check that a variable term being unified with a second term does not occur within that second term. It is necessary to ensure the algorithm terminates and that the unification procedure is correct.
- The two terms "X" and "f(X)" are not [unifiable]: there is no substitution for X that makes the two terms equal.
- If a term t has a complex structure, checking whether X occurs in t can be very inefficient.
- Prolog does NOT use the occur-check: not sound !!

OCCUR CHECK: EXAMPLE

- **Example:** Consider the terms:
  - `p(X,f(X)).`
  - `p(Y,Y).`
  - Without the occur check they unify, and the substitution produced (containing an infinite term) is: `Y=f(f(f(.....)))`
- **So if p represented the predicate "greater than" and f the successor function, it would follow that there exists a number greater than itself!**
- **Unsound proof!!**

THE RESOLUTION PRINCIPLE FOR GENERAL CLAUSES

- Clauses in which variables may occur.
- Let C_1 e C_2 be two clauses of the form:

  $C_1 = A_1 \vee ... \vee A_n$ $\qquad\qquad$ $C_2 = B_1 \vee ... \vee B_m$

- where A_i (i=1..n) and B_j (j=1..m) is a positive or negative literal in which variables may occur.
- If there exist A_i and Bj such that $[A_i]\theta = [\sim B_j]\theta$, where θ is the most general unifying substitution, then a new clause C_3 (the resolvent) can be derived:

  $[A_1 \vee .. \vee A_{i-1} \vee A_{i+1} \vee .. \vee A_n \vee B_1 \vee ... \vee B_{j-1} \vee B_{j+1} \vee .. \vee B_m]\theta$

- Given two clauses C_1 e C_2, their resolvent C_3 is a logical consequence of $C_1 \cup C_2$.

MGU FOR DERIVING THE RESOLVENT
Inference rule

- **Example**

From the following clauses:

$p(a,b) \vee q(X,a) \vee c(X)$ (C1)

$\sim p(a,Y) \vee \sim c(f(Y))$ (C2)

the following resolvents are obtained:

$q(X,a) \vee c(X) \vee \sim c(f(b))$ (C3)

$p(a,b) \vee q(f(Y),a) \vee \sim p(a,Y)$ (C4)

C3: obtained by selecting p(a,b) from C1 and ~p(a,Y) from C2 and applying the mgu $\theta=\{Y/b\}$.

C4: obtained by selecting c(X) from C1 and ~c(f(Y)) from C2 and applying the mgu $\theta=\{X/f(Y)\}$.

EXAMPLE

- **Consider the sets:**

H= {$\forall$X (man(X) $\rightarrow$ mortal(X)), man(socrates)}

F = {$\exists$X mortal(X)}

Transforming H and ~F into clauses gives:

H^C = {~man(X) $\vee$ mortal(X), man(socrates)}

F^C = {~mortal(X)}

The set $H^C \cup F^C$ is as follows:

{~man(X) $\vee$ mortal(X), (1)

man(socrates), (2)

~mortal(Y)} (3)

The variable X of F^C is renamed to Y, which does not appear in any other clause of the set. This operation is performed every time new resolvents are added to the clause set.

EXAMPLE

{~man(X) ∨ mortal(X), (1)

man(socrates), (2)

~mortal(Y)} (3)

- At step 1, all the possible resolvents, and the most general unifying substitutions applied to derive them, are:

{mortal(socrates), (4) θ={X/socrates}

~man(Z)} (5) θ={X/Y}

- At step 2, from (2) and (5), the empty clause is derived by applying the substitution {Z/socrates}. The empty clause can also be derived from clauses (3) and (4) by applying the substitution {Y/socrates}.

EXAMPLE

1. dog(fido).
2. ~barks(fido).
3. wags_tail(fido).
4. meows(geo).
5. wags_tail(X) and dog(X) -> friendly(X).
6. friendly(X1) and ~ barks(X1) -> ~scared(Y1,X1).
7. dog(X2) -> animal(X2).
8. meows(X3) -> cat(X3).

Goal: 9. there exist X4, Y dog(X4) and cat(Y) and ~scared(Y,X4).

TRANSLATION INTO CLAUSES

1. dog(fido).
2. ~barks(fido).
3. wags_tail(fido).
4. meows(geo).
5.~wags_tail(X) or ~dog(X) or friendly(X).
6.~friendly(X1) or barks(X1) or ~scared(Y1,X1).
7.~dog(X2) or animal(X2).
8.~meows(X3) or cat(X3).

Negated goal: 9.~dog(X4) or ~cat(Y) or scared(Y,X4).

EXAMPLE

1. dog(fido).
2. ~barks(fido).
3. wags_tail(fido).
4. meows(geo).
5.~wags_tail(X) or ~dog(X) or friendly(X).
6.~friendly(X1) or barks(X1) or ~scared(Y1,X1).
7.~dog(X2) or animal(X2).
8.~meows(X3) or cat(X3).

Negated goal: 9.~dog(X4) or ~cat(Y) or scared(Y,X4).

From 9 and 1: 10. ~cat(Y) or scared(Y,fido).

From 10 and 6: 11. ~friendly(fido) or barks(fido) or ~cat(Y).

From 11 and 2: 12. ~friendly(fido) or ~cat(Y).

From 8 and 12: 13. ~friendly(fido) or ~meows(Y).

From 13 and 4: 14. ~friendly(fido).

From 5 and 14: 15. ~wags_tail(fido) or ~dog(fido).

From 3 and 15: 16. ~dog(fido)

From 1 and 16. CONTRADICTION !!

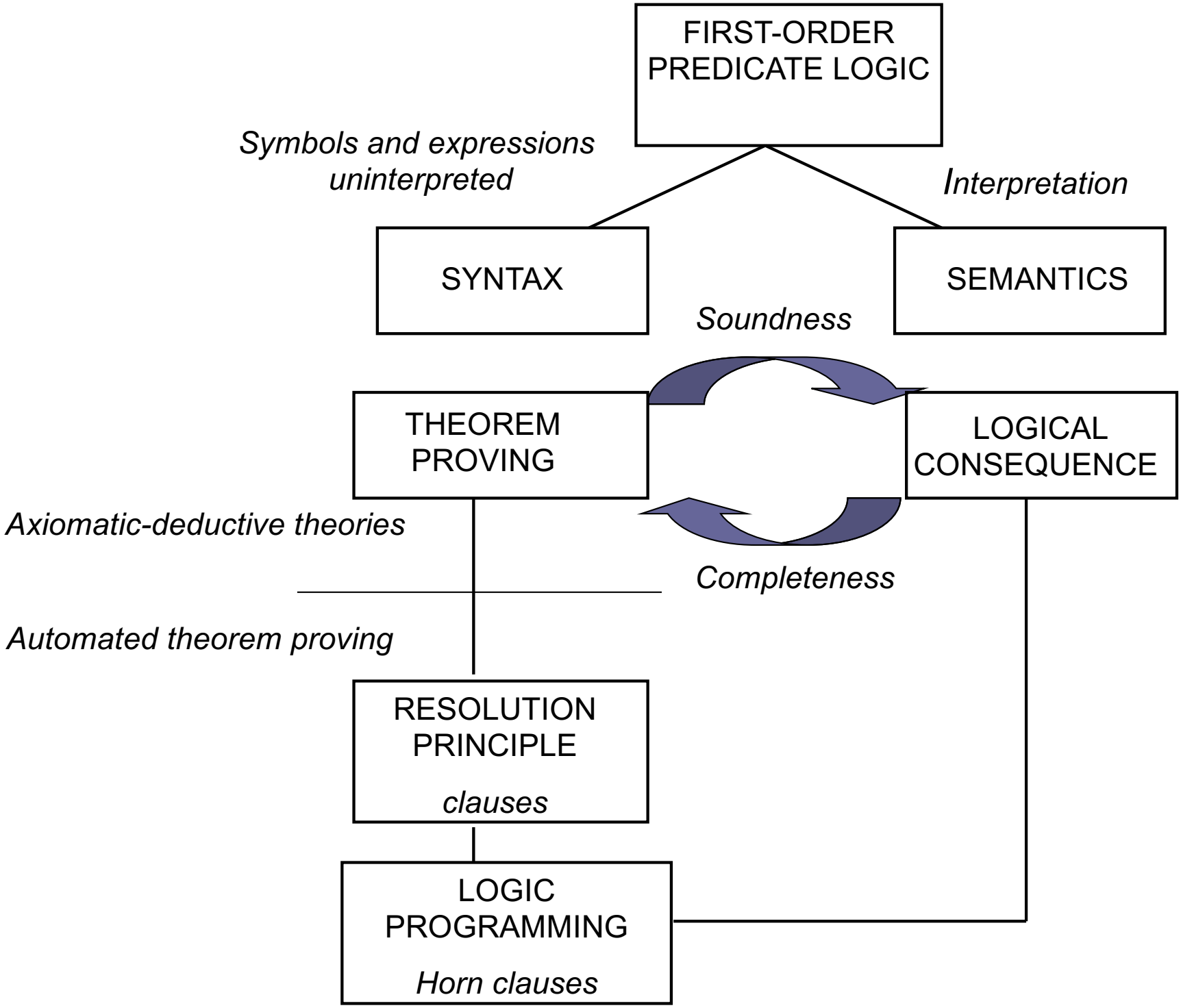
SOUNDNESS and COMPLETENESS

- It can be shown that, under suitable strategies, resolution is **sound and complete**.
- If the empty clause is generated, the theory $H^C \cup F^C$ is unsatisfiable, and if the theory $H^C \cup F^C$ is unsatisfiable the derivation generates the empty clause in a finite number of steps.

Theorem: (Soundness and completeness of resolution)

A set of clauses is unsatisfiable if and only if the resolution algorithm terminates successfully in a finite number of steps, generating the empty clause.

- The resolution method proceeds exhaustively, generating all possible resolvents at each step.


FIRST-ORDER PREDICATE LOGIC
Symbols and expressions uninterpreted
Interpretation
SYNTAX
SEMANTICS
Soundness
THEOREM PROVING
LOGICAL CONSEQUENCE
Axiomatic-deductive theories
Completeness
Automated theorem proving
RESOLUTION PRINCIPLE
clauses
LOGIC PROGRAMMING
Horn clauses

HORN CLAUSES

- Horn-clause logic is a subset of clause logic
- **Horn clauses have at most one positive literal.**
- Clauses can be written in an equivalent form by replacing the disjunction symbol $\vee$ and negation ~ with the logical implication symbol ($\rightarrow$) recalling that:

  $\sim B \vee A$ is equivalent to $B \rightarrow A$

- In what follows, the logical implication $B \rightarrow A$ will be denoted by:

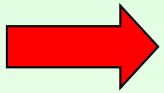
  $A \leftarrow B$ ("B implies A", or "A if B").

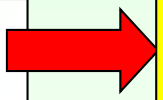
- The clause: $A_1 \vee \ldots \vee A_n \vee \sim B_1 \vee \ldots \vee \sim B_m$
- can be rewritten as: $A_1, \ldots, A_n \leftarrow B_1, \ldots, B_m$
- where the "," symbols separating the atoms A_i are to be read as disjunctions, while those separating the atoms B_j are conjunctions.
- Horn clauses: $A \leftarrow B_1, \ldots, B_m$ $\leftarrow B_1, \ldots, B_m$ Goal

Expressive Power of Horn Clauses

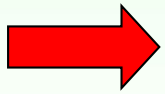
- Many predicate-logic formulas can be written as Horn clauses.

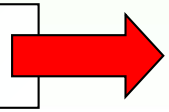
- Examples:

$\forall x\ cat(x) \vee dog(x) \rightarrow pet(x)$  $pet(x) \leftarrow cat(x)$, $pet(x) \leftarrow dog(x)$

$\forall x\ poodle(x) \rightarrow dog(x) \wedge small(x)$  $dog(x) \leftarrow poodle(x)$, $small(x) \leftarrow poodle(x)$

- BUT:

$\forall x\ human(x) \rightarrow male(x) \vee female(x)$  ????

$\forall x\ dog(x) \wedge \sim abnormal(x) \rightarrow has_4_legs(x)$  ????

EXAMPLE NOT EXPRESSIBLE AS HORN CLAUSES

$H=\{on(b1,table) \wedge on(b2,table)) \wedge ((color(b1,red) \vee color(b2,red)))\}$

is expressible as clauses, but not as Horn clauses.

- Its transformation into clauses is in fact:

  $H^C=\{on(b1,table), on(b2,table), color(b1,red) \vee color(b2,red)\}$

  where the clause: $color(b1,red) \vee color(b2,red)$
  is not a Horn clause.

- Resolution for Horn clauses: **SLD resolution** *resolution with **S**election rule, **L**inear input strategy for **D**efinite clauses*

- SLD resolution works by contradiction, so we proceed by negating the formula F to be proved.

- Since F is a conjunction of existentially quantified atomic formulas, its negation will produce a universally quantified disjunction of negated atomic formulas, i.e., a Horn goal clause.